

Lecture 22: Introduction to Machine Learning

Heather Mattie, PhD

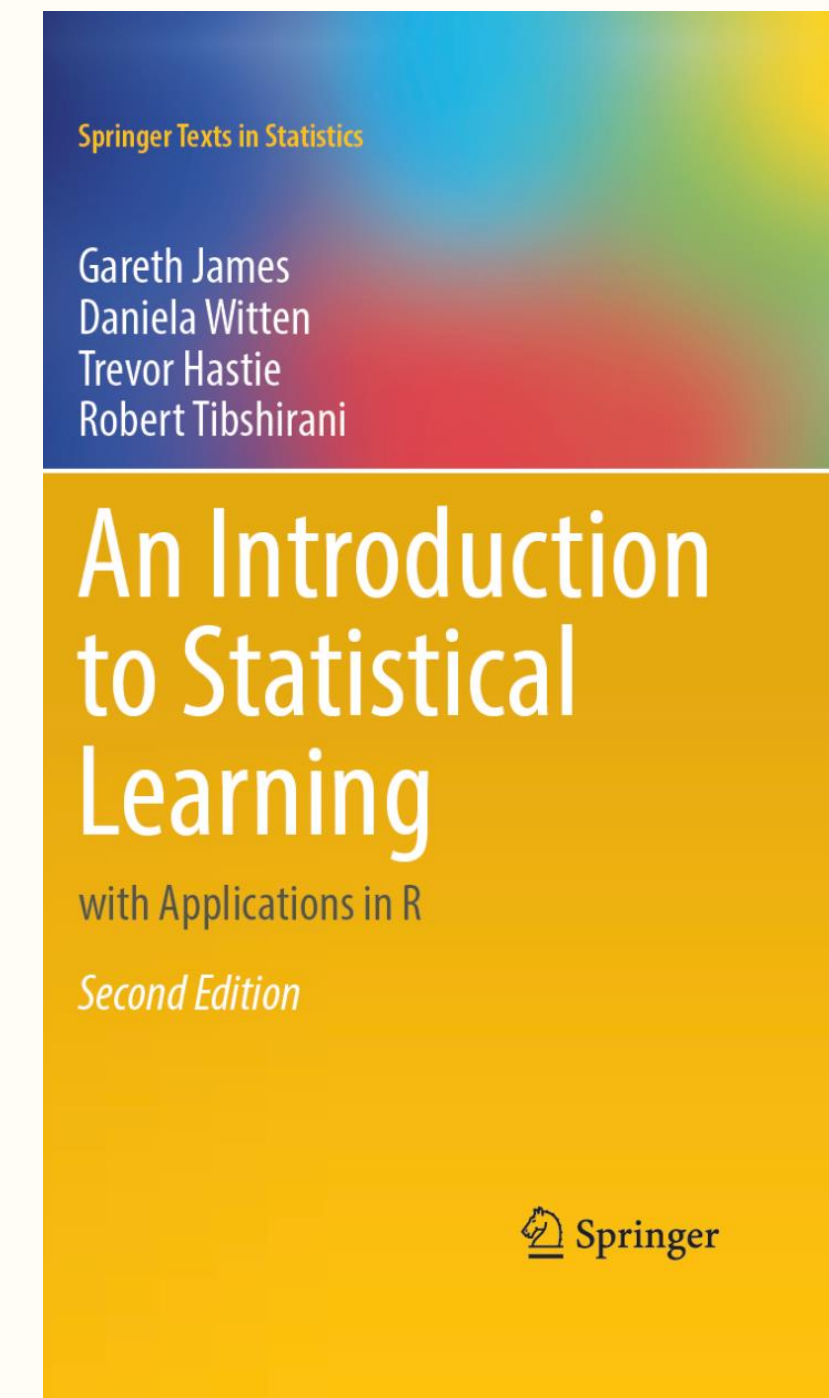
November 18, 2025

Recipe of the Day!

Simply the Best Blueberry Pie



An Introduction to Statistical Learning (ISLR)



Announcements

- Heather's office hour moved to Wednesday, 1-2pm on Zoom this week
- Lab this week on machine learning
- Project preference form due Friday, November 21st
- Homework #4 due Friday, December 5th
- Homework #5 due Friday, December 12th
- No class November 25th and 27th

Introduction to Machine Learning

Heather Mattie, PhD



Intro to ML Roadmap

1. Machine learning fundamentals
2. Evaluating model performance
3. Logistic regression

Introduction to Machine Learning

Machine learning fundamentals

Heather Mattie, PhD

The Data Science Pipeline



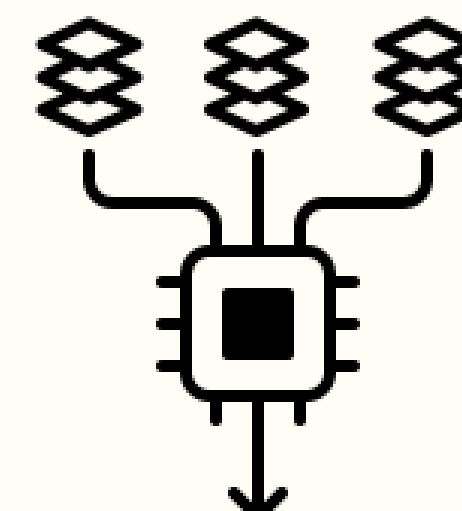
**Collecting
and
importing
(loading)
the data**



**Processing
(cleaning,
wrangling)
the data**



**Visualizing
and
summarizing
the data**



**Building
models
(statistical
and ML)**

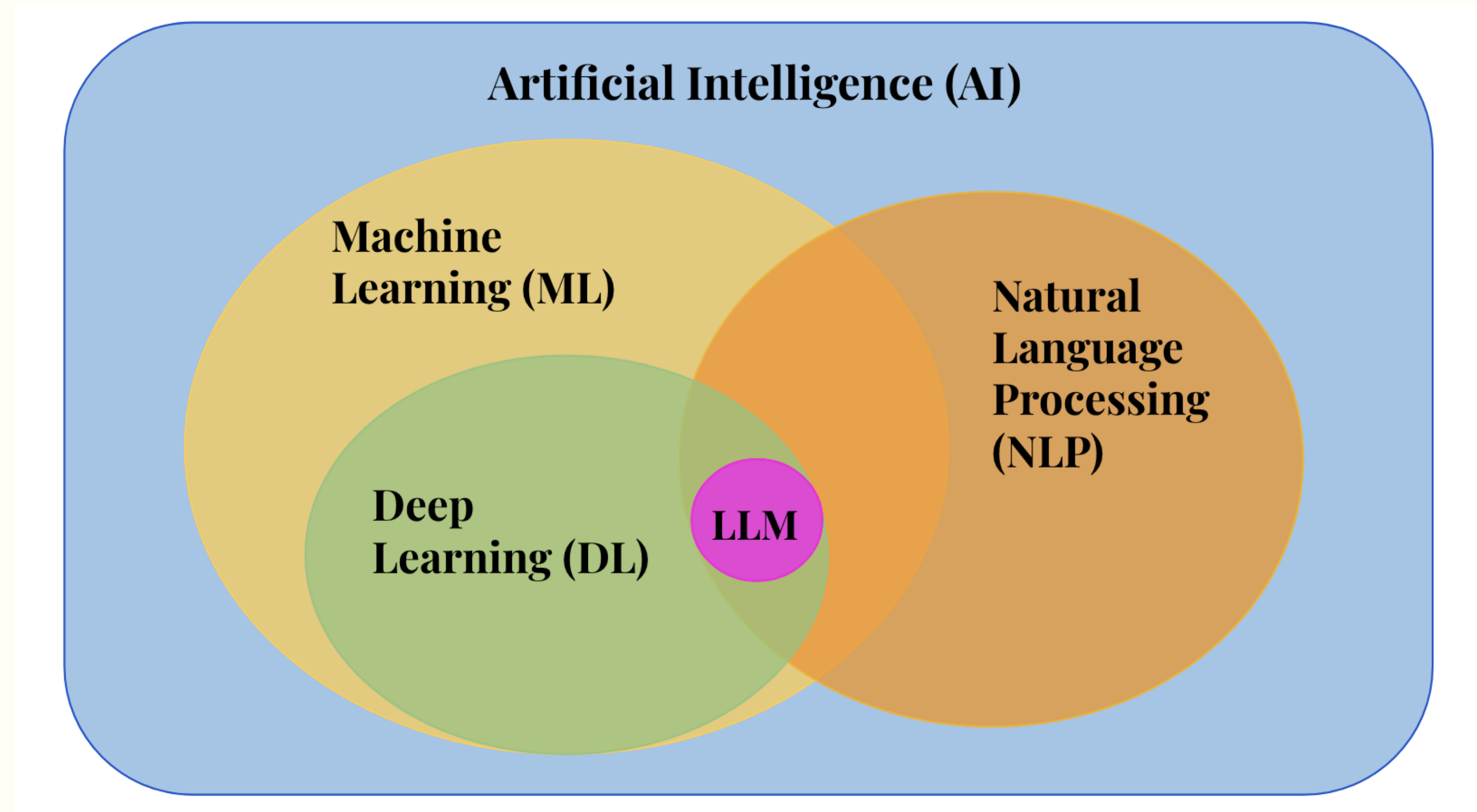


**Interpretation
and
communication
of results**

Machine learning

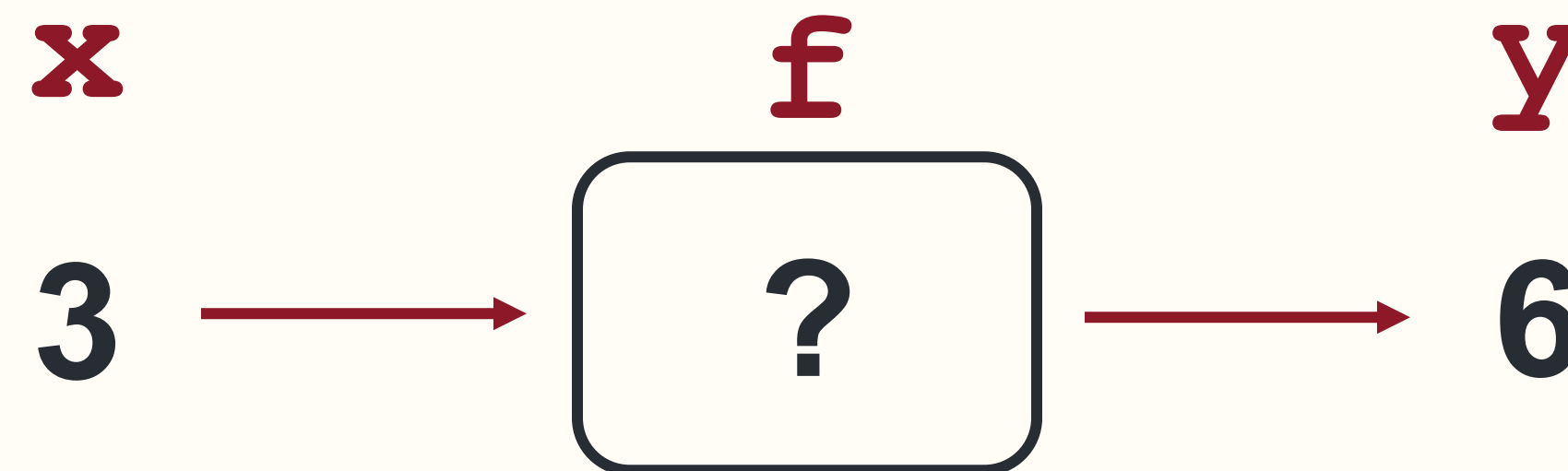
Machine learning (ML), also known as **statistical learning**, is a subfield of AI that allows computer systems to learn from data and improve their performance over time.

- Algorithms analyze large datasets, identify patterns, and make predictions based on those patterns



ML intuition

To help understand how a machine “learns”, try to guess the function that takes x as an input and generates y as an output. The first values are $x = 3$ and $y = 6$.

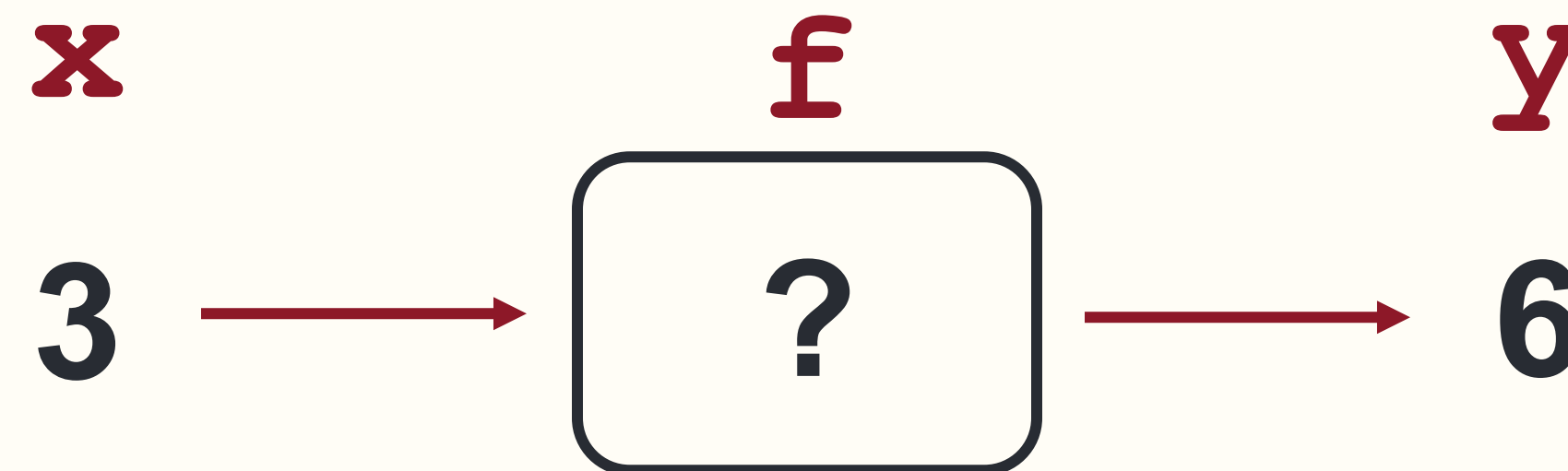


ML intuition

To help understand how a machine “learns”, try to guess the function that takes x as an input and generates y as an output. The first values are $x = 3$ and $y = 6$.

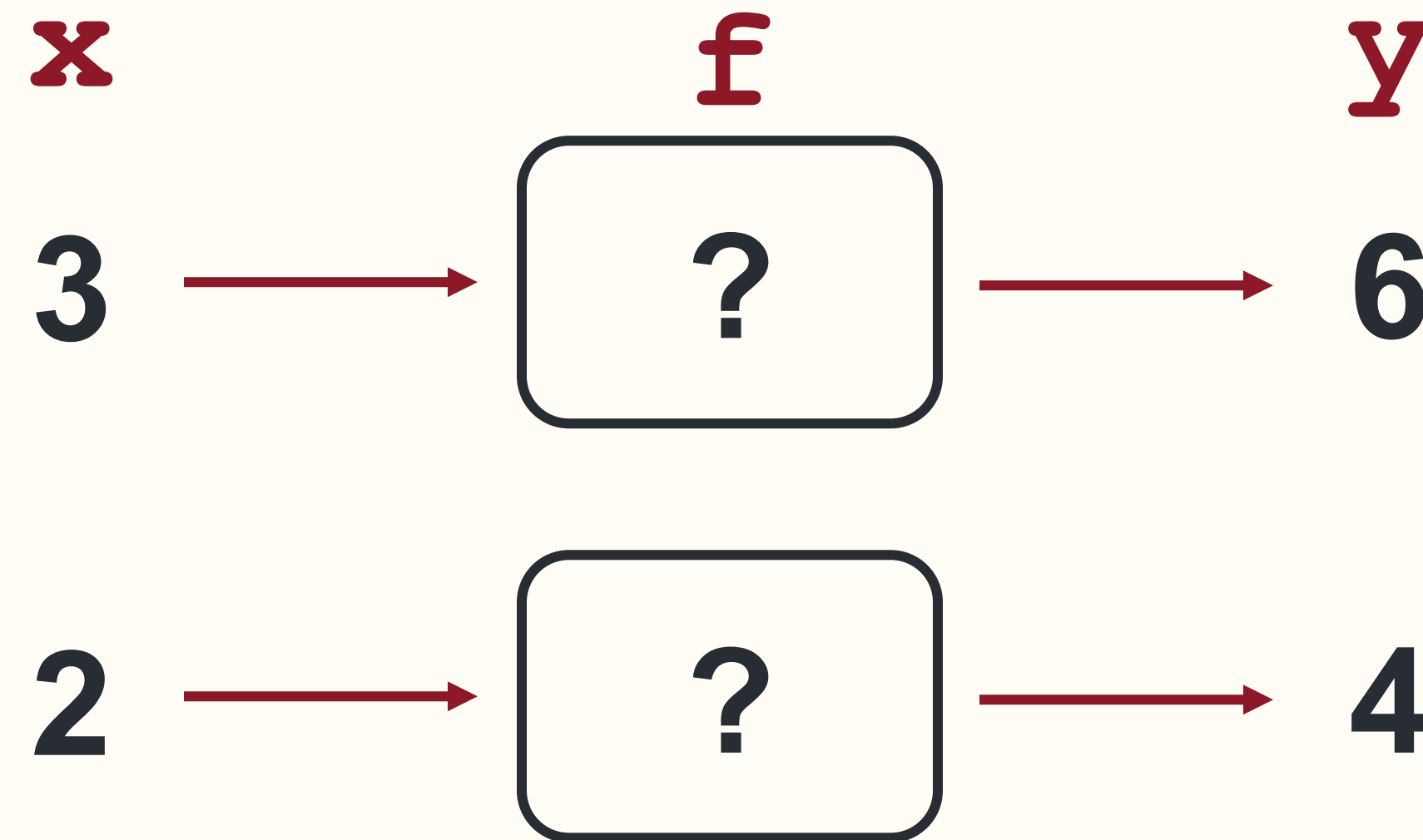
A couple of options for the function are:

$$y = x + 3$$
$$y = 2x$$



ML intuition

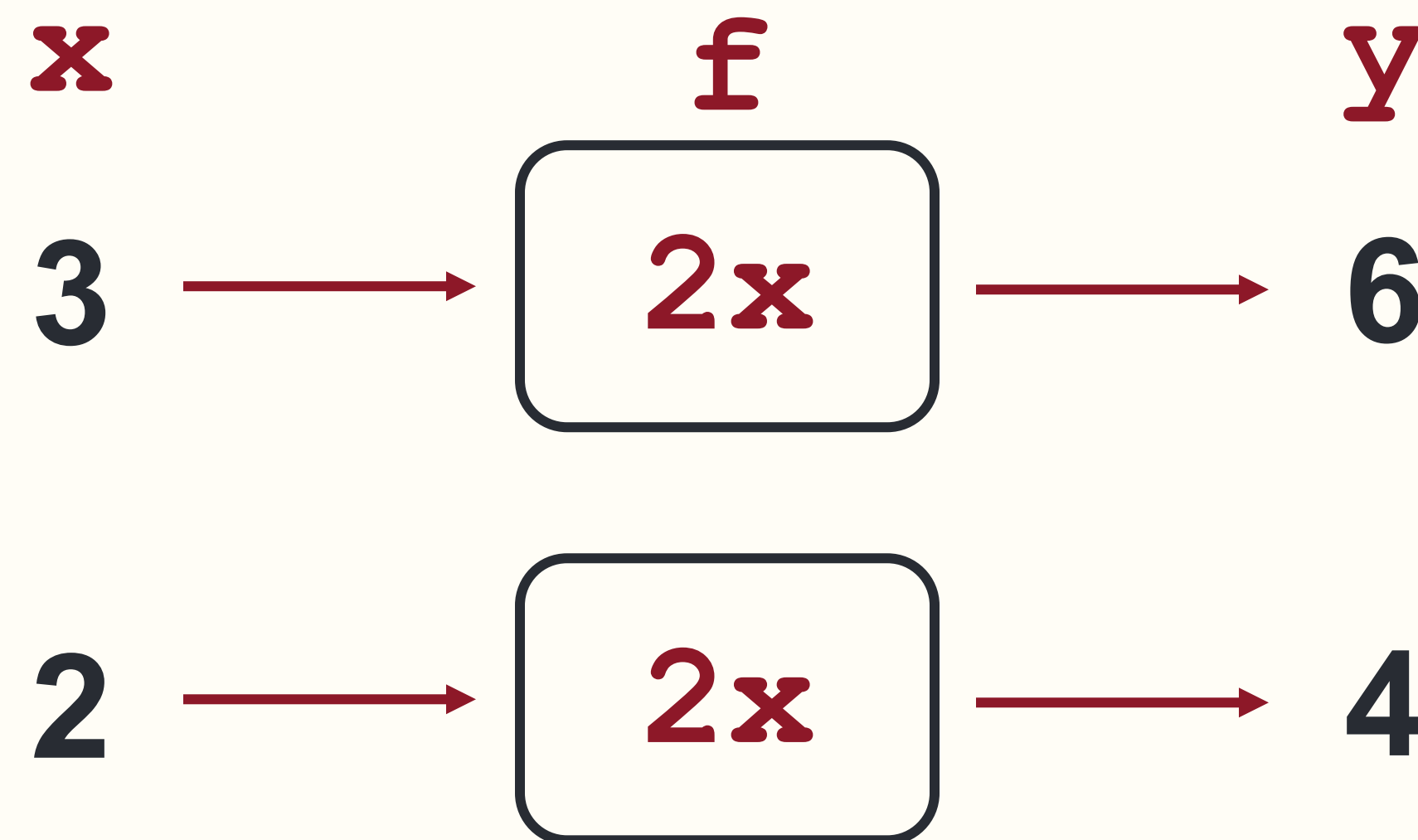
Since there are multiple options, use another input and output. This should give more information about what f is.



ML intuition

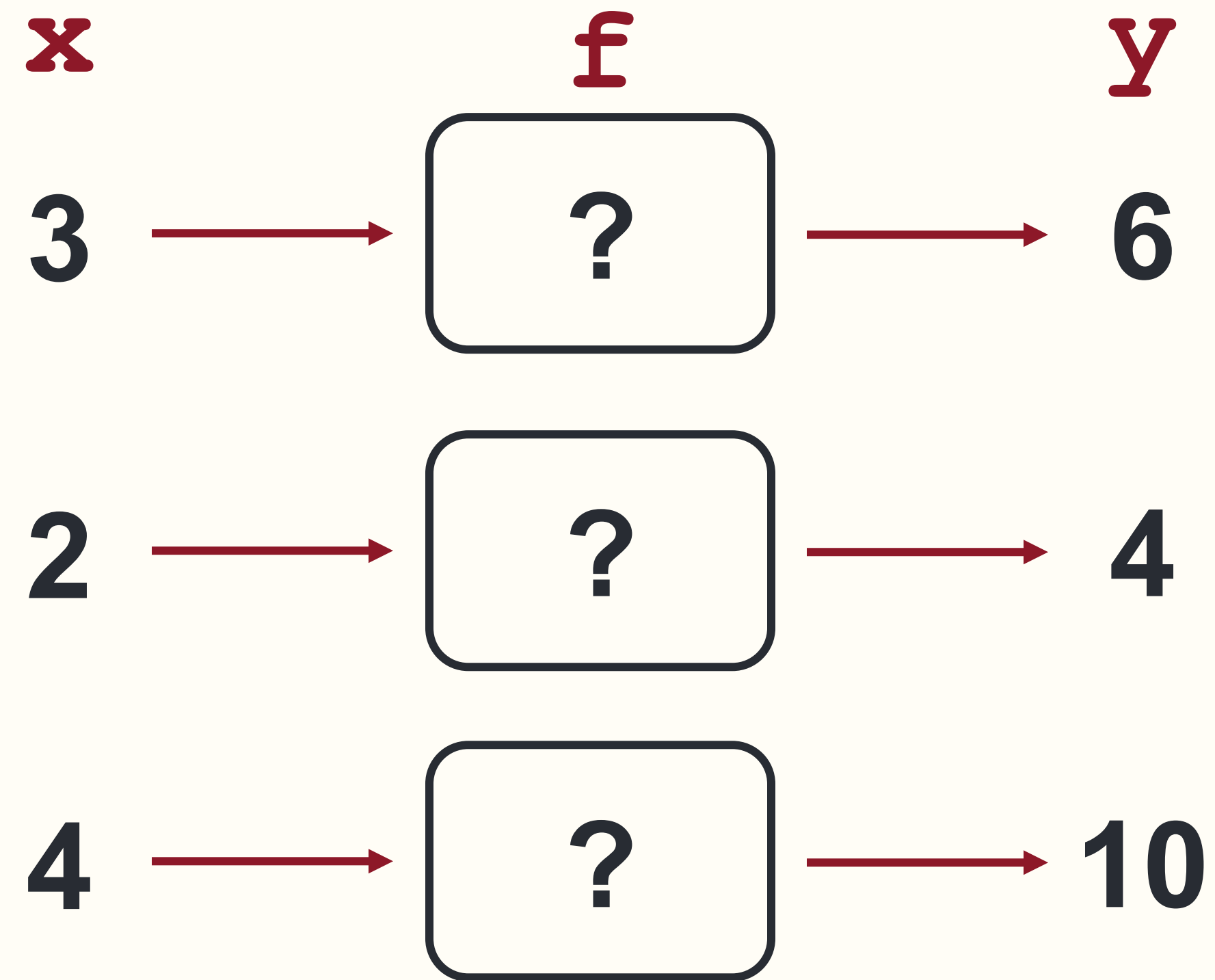
Since there are multiple options, use another input and output. This should give more information about what f is.

The function $y = 2x$ would work for both.



ML intuition

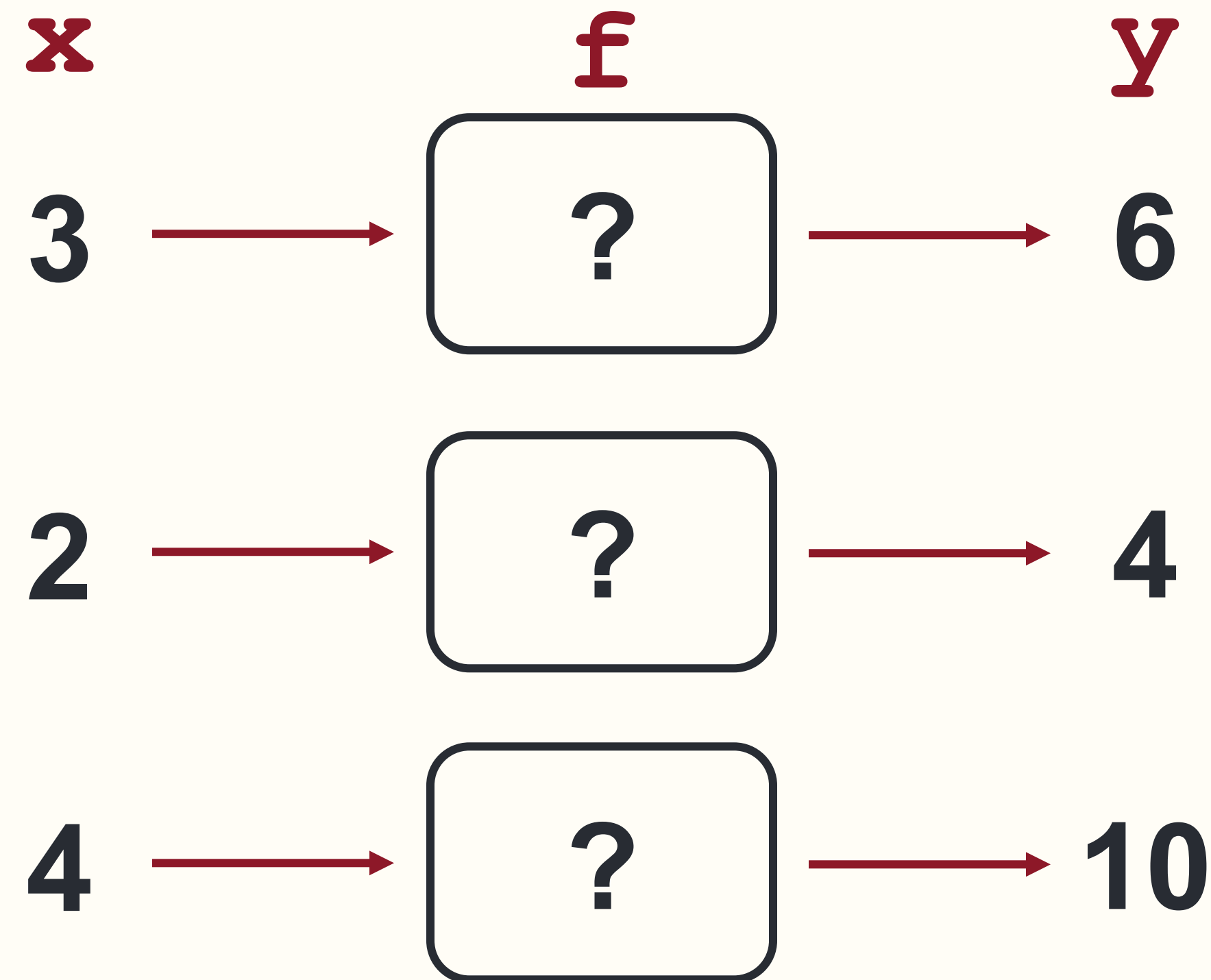
The function $y = 2x$ doesn't work when a third input and output pair are introduced.



ML intuition

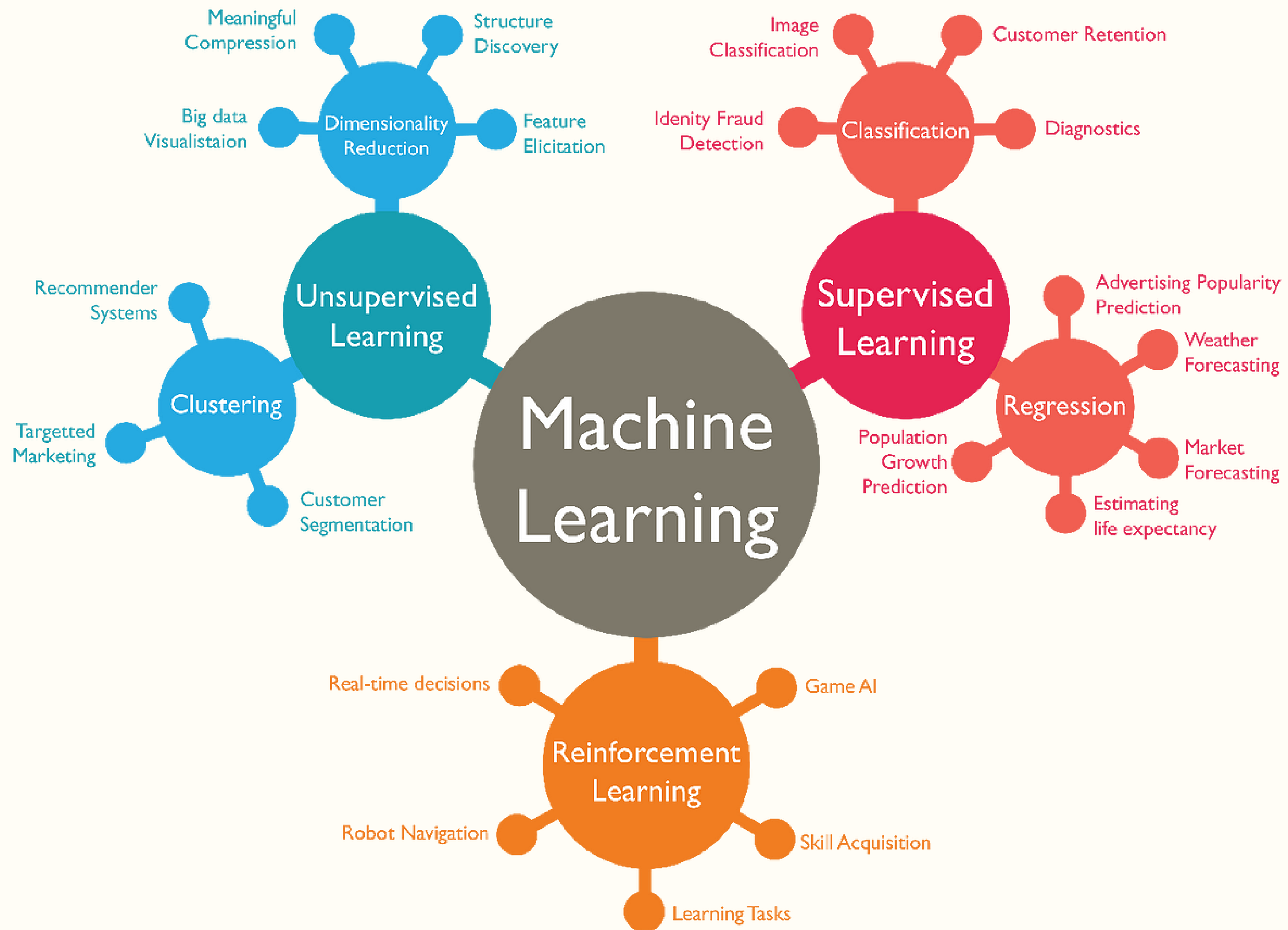
The function $y = 2x$ doesn't work when a third input and output pair are introduced.

The function is more complicated and more data is needed.



Types of ML

1. **Supervised learning**: the algorithm learns from a completely labeled dataset (features and outcome values are known).
2. **Unsupervised learning**: the algorithm learns from a dataset with known feature values, but unknown outcome values.
3. **Reinforcement learning**: the algorithm trains an agent to make a sequence of decisions by interacting with an environment through trial and error.



Terminology

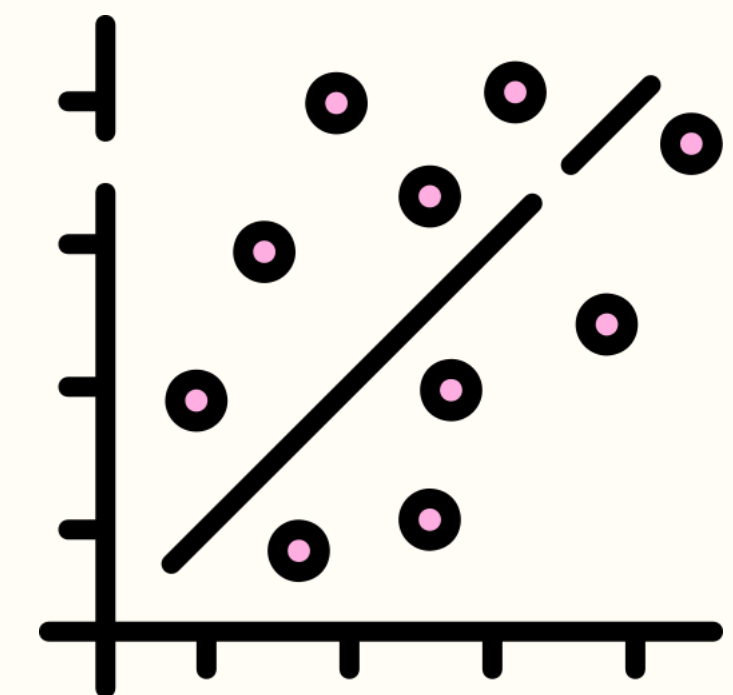
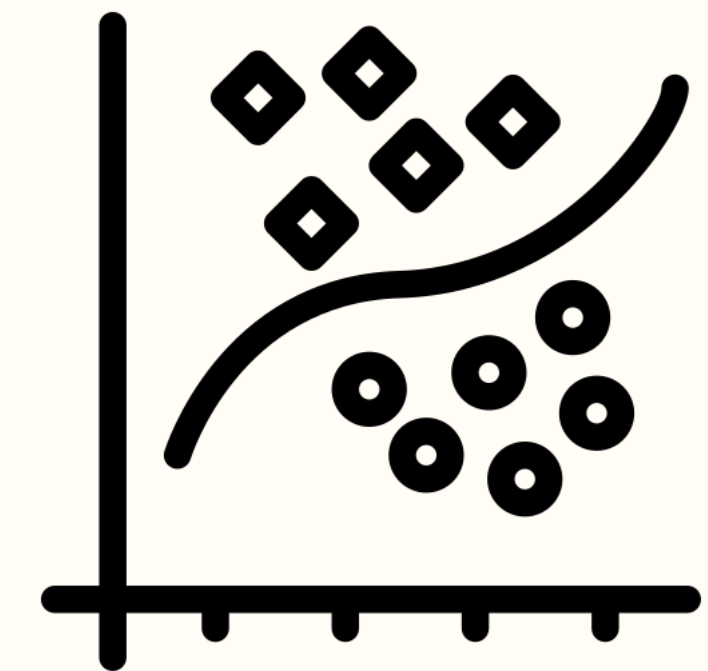
Machine learning often uses different terminology than traditional statistics, but the underlying ideas are very similar.

- **Outcome (label, target, Y)**: what the model is trying to predict
- **Features (predictors, $X_1, \dots, X_p$)**: variables used to make predictions
- **Algorithm (model)**: the prediction model being trained

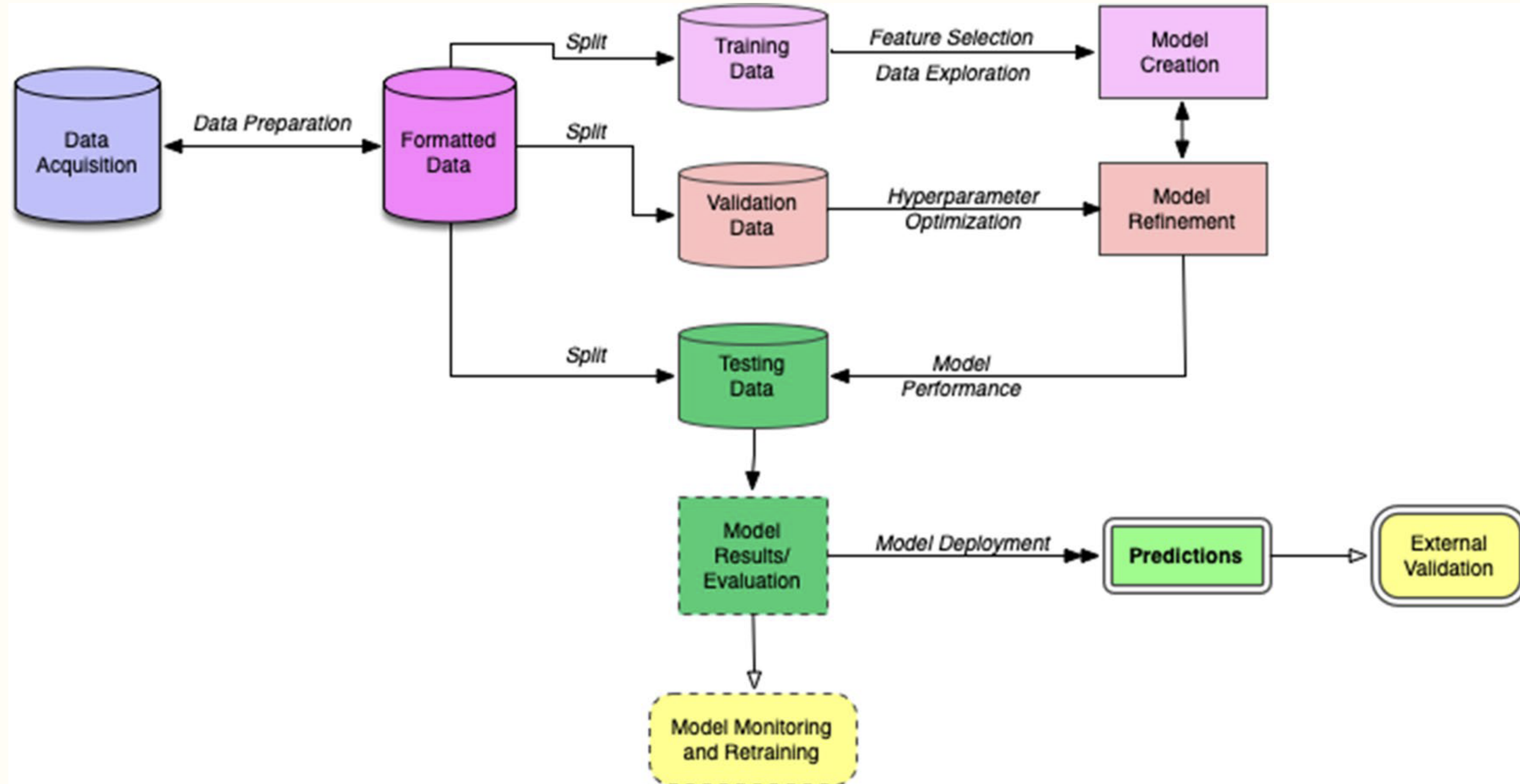
Types of tasks

The type of prediction depends on the **task**.

- **Classification**: Y is **categorical** (or binary)
 - The task is to predict the correct **class** (category)
 - The prediction is either correct or incorrect
- **Regression**: Y is **continuous**
 - The task is to predict a continuous number, labeled $\hat{y}$
 - The error of the prediction is $y - \hat{y}$



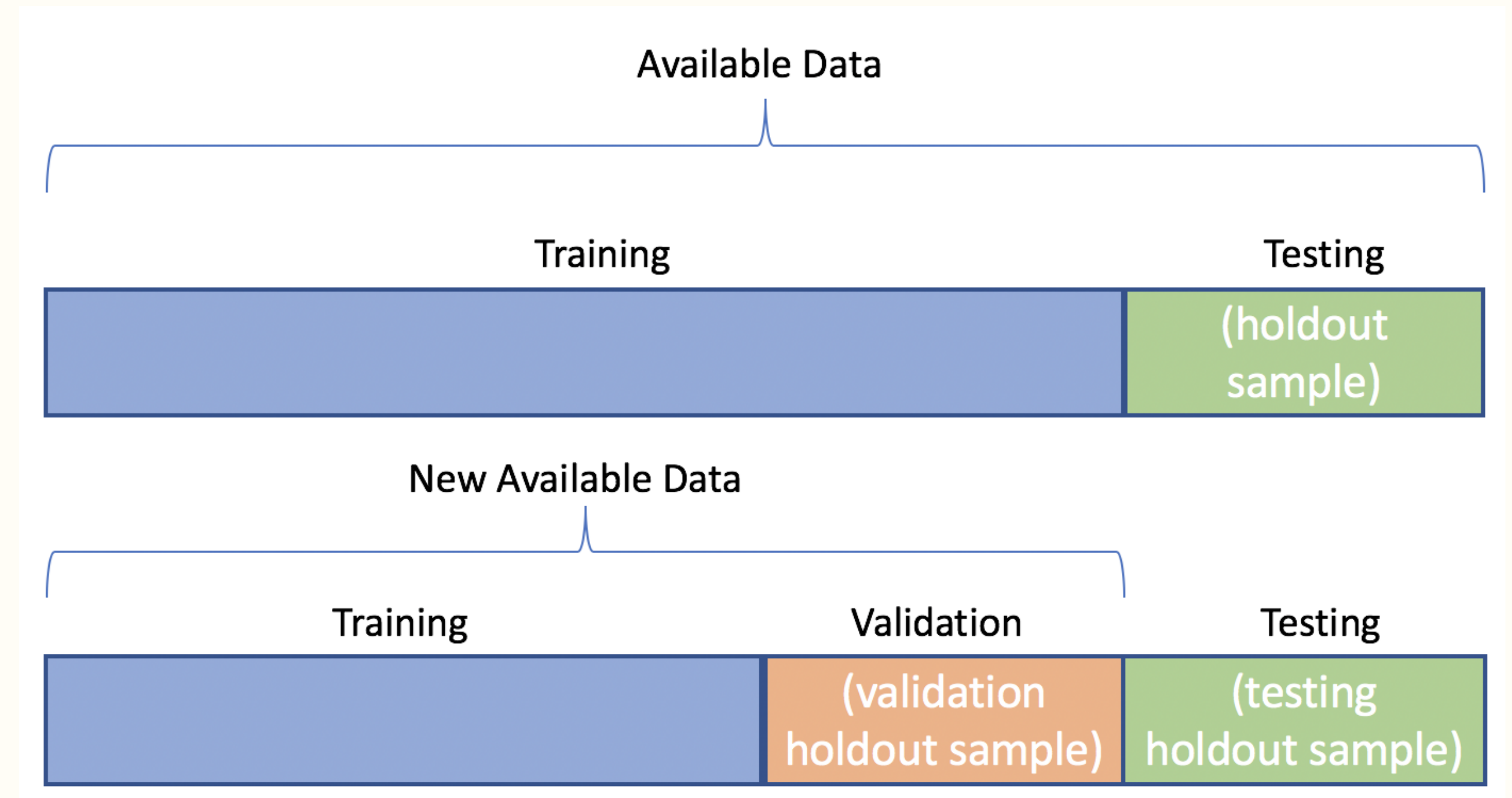
ML workflow



Training and test sets

A dataset must be split into partitions for training and testing.

- **Training set:** data used to train the model (learn patterns)
- **Validation set:** data used to tune hyperparameters of the model (maximize performance)
- **Test set:** data used for final, unbiased performance evaluation



Summary

- ML models learn patterns from data by minimizing error or maximizing predictive accuracy.
- Supervised learning uses labeled outcomes; unsupervised learning discovers structure without labels.
- Classification predicts categories and regression predicts numeric/continuous values.
- Training, validation, and test sets support model fitting, tuning, and unbiased performance assessment.

Introduction to Machine Learning

Evaluating model performance

Heather Mattie, PhD

Why evaluate ML models?

ML models must be evaluated in order to:

- Determine how well the model generalizes to new data
- Compare models and select the best performer
- Identify bias, overfitting, and underfitting
- Support safe and effective deployment in healthcare settings

The type of machine learning and **task** dictate how model performance is measured (quantified).

Performance metrics must always align with the problem (task) being solved.

Performance for supervised learning

With supervised learning all labeled outcomes are known.

Performance measures **how close the predictions are to true labels.**

- Classification: overall accuracy, sensitivity (recall), specificity, AUROC, calibration metrics, etc.
- Regression: mean absolute error (MAE), mean squared error (MSE), root mean squared error (RMSE), etc.

Performance for unsupervised learning

With unsupervised learning, labeled outcomes are unknown.

Performance evaluates **structure or coherence of discovered patterns**.

- Often more subjective in nature
- Application-dependent
- Metrics include clustering metrics (e.g., clinical interpretability) and dimensionality reduction metrics (e.g., explained variance)

Binary classification

When performing binary classification, the outcome can only have one of two values (usually indicated by a 0 or 1).

A 2x2 table can be used to show which values the model accurately predicts and which are incorrect.

	Actual positive	Actual negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

Binary classification metrics

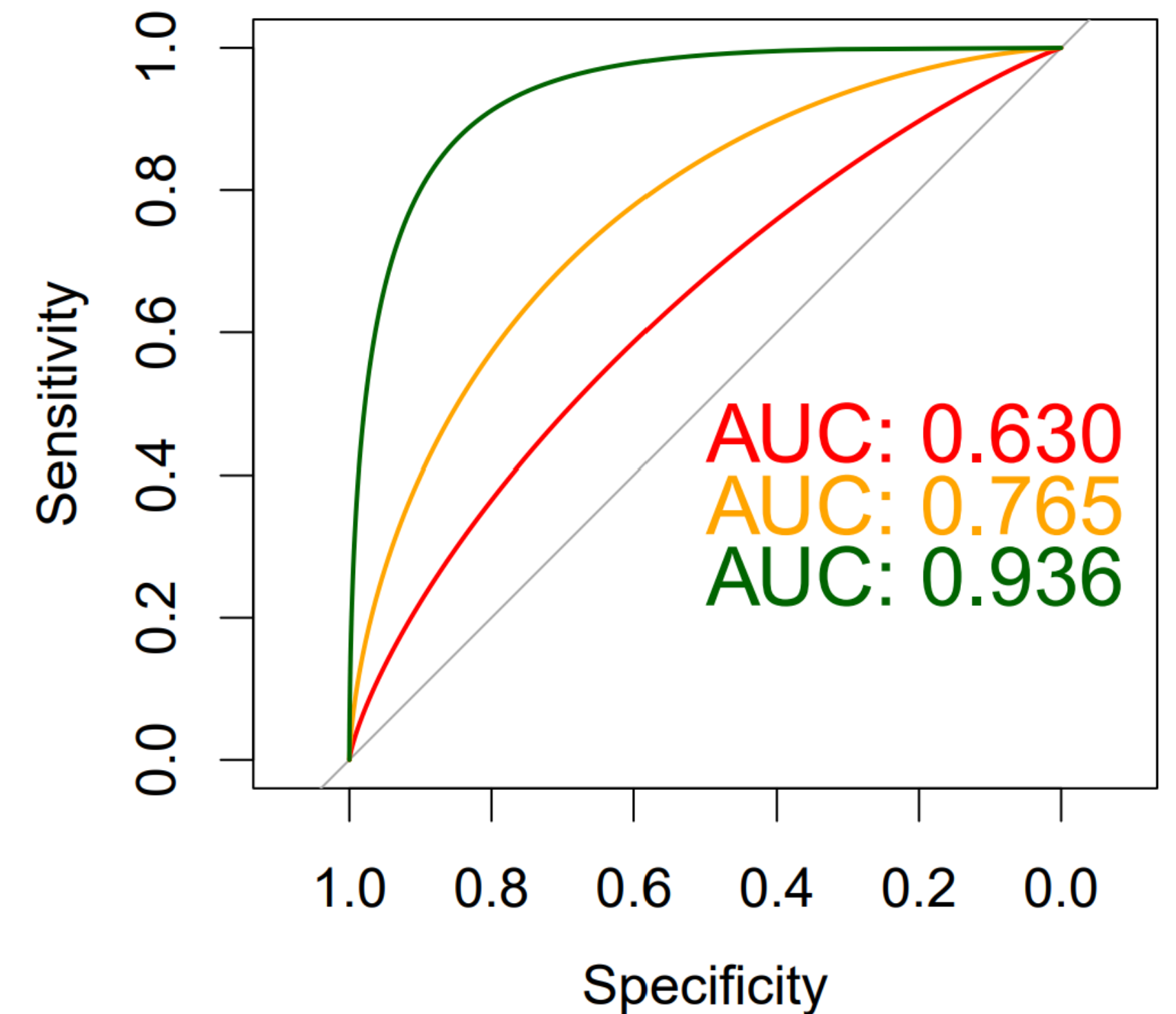
		Actual positive	Actual negative
	Predicted positive	True positives (TP)	False positives (FP)
	Predicted negative	False negatives (FN)	True negatives (TN)

- **Overall accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Sensitivity (recall):** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **Precision:** $TP / (TP + FP)$
- **F₁ score:** $2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$
 $= TP / (TP + 0.5(FP + FN))$

Binary classification metrics

The **receiver operating curve (ROC)** is a graph that plots the model's sensitivity on the y-axis and 1-specificity on the x-axis for various classification thresholds.

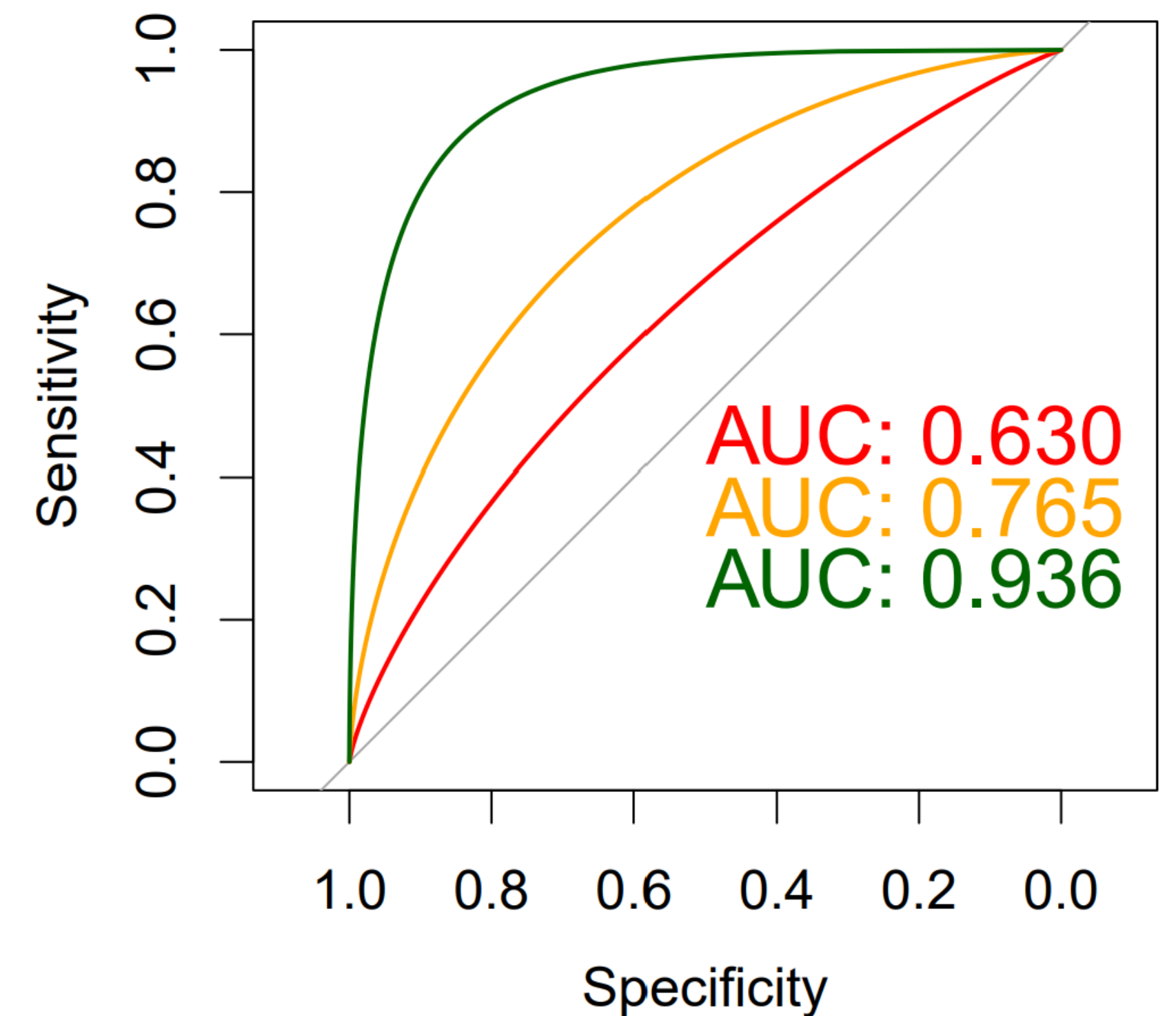
- The classification threshold determines if predicted probabilities are transformed into a 0 or a 1.



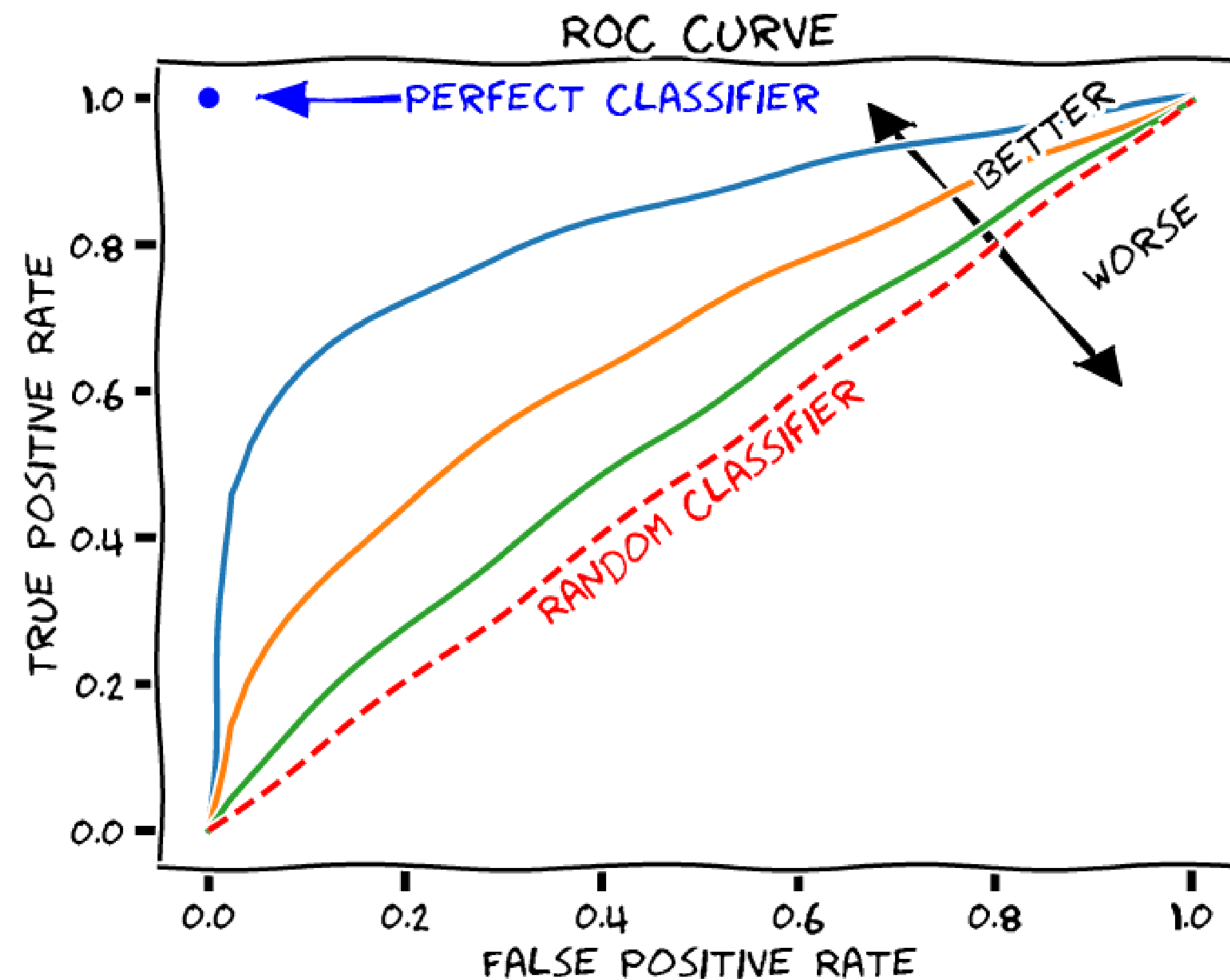
Binary classification metrics

The **area under the receiver operating curve (AUROC)** summarizes the ROC curve into a single number.

- A value closer to 1 indicates a better performing model
- A value of 0.5 indicates a model that is as good as random guessing



ROC and AUROC



Summary

- ML performance metrics must align with the problem type and data characteristics.
- Binary classification models are evaluated using metrics such as accuracy, sensitivity, specificity, precision, F_1 score, and AUC.
- Confusion matrices provide insight into false positives/negatives and classification trade-offs of a model.

Introduction to Machine Learning

Logistic regression

Heather Mattie, PhD

Binary classification

Many public health decisions often hinge on yes/no outcomes that directly guide clinical or policy actions. This makes binary classification ML models very common and useful.

Many core public health outcomes are binary:

- Disease vs. no disease
- Hospitalized vs. not hospitalized
- Screen-positive vs. screen-negative
- Treatment success vs. failure

Binary classification

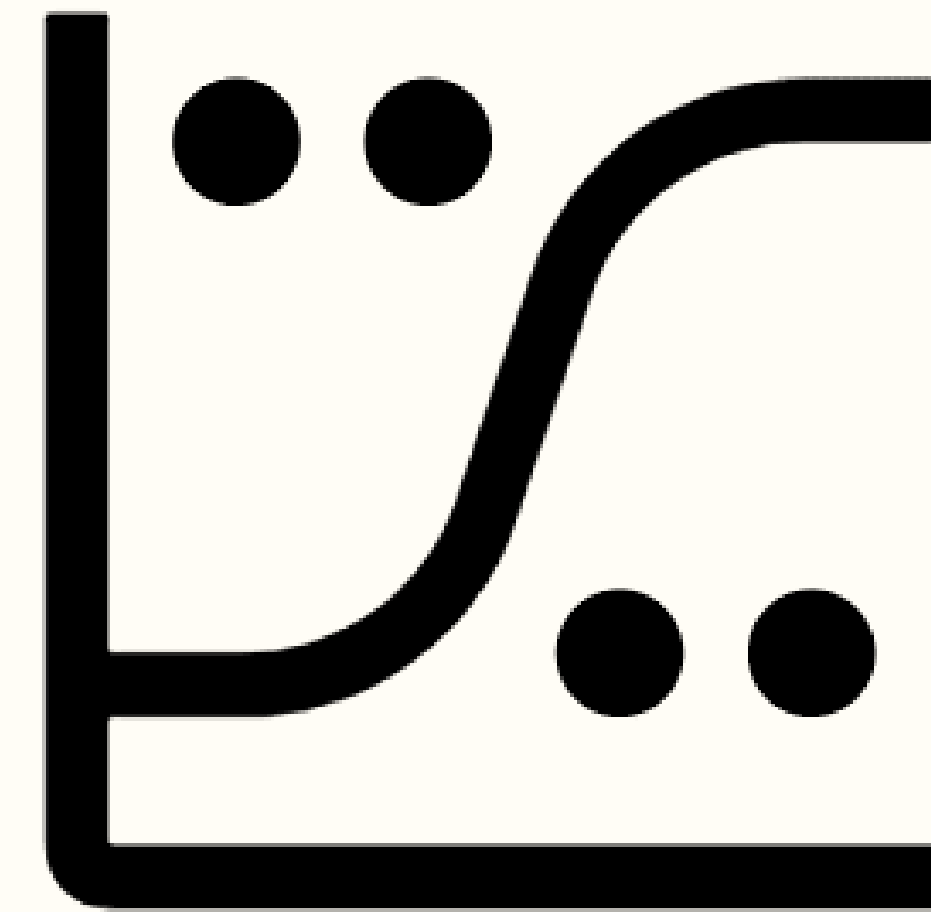
Binary classification ML models can:

- Support risk stratification and triage by identifying high-risk individuals.
- Enable early detection and prevention strategies (e.g., predicting diabetes, hypertension, HIV acquisition).
- Simplify communication of results to clinicians, policymakers, and patients.
- Work well with structured EHR/claims data, which often include categorical risk indicators.
- Facilitate evaluation using intuitive metrics like sensitivity, specificity, and AUC, aligned with clinical decision-making.

Logistic regression

Logistic regression is a foundational modeling approach for predicting binary outcomes, making it especially valuable in public health and clinical research.

- Models the probability that an event occurs:
 $p = P(Y = 1)$
- Predicts $Y = 1$ when $p >$ some threshold



Case study: melanoma

The **boot** package contains the **melanoma** dataset which records tumor thickness (mm) and survival status after tumor removal for 205 patients with malignant melanoma.

- Focus only on patients who either died from melanoma (1) or were still alive at the end of the study (2); drop all who died of other causes (3).

Case study: melanoma

```
library(tidyverse)
library(boot)
data(melanoma)

melanoma <- melanoma %>%
  filter(status %in% c(1, 2)) %>%
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
head(melanoma)
```

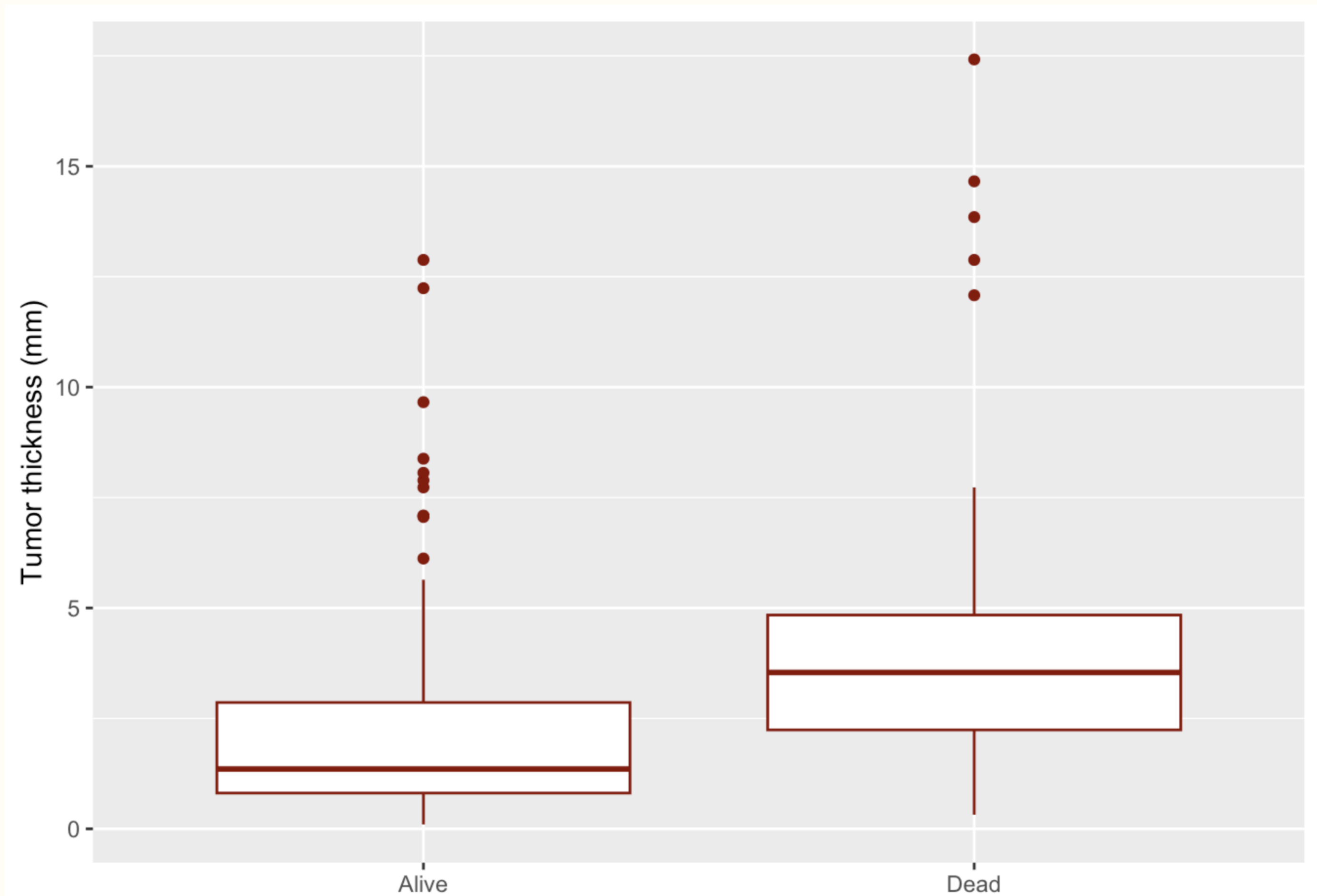
##	time	status	sex	age	year	thickness	ulcer	y
## 3	35	Alive	1	41	1977	1.34	0	0
## 5	185	Dead	1	52	1965	12.08	1	1
## 6	204	Dead	1	28	1971	4.84	1	1
## 7	210	Dead	1	77	1972	5.16	1	1
## 9	232	Dead	1	49	1968	12.88	1	1
## 10	279	Dead	0	68	1971	7.41	1	1

Case study: melanoma

```
table(melanoma$status)
```

```
##  
## Alive   Dead  
##   134    57
```

```
melanoma %>%  
  ggplot(aes(status, thickness)) +  
  geom_boxplot(color = "darkred") +  
  xlab("") +  
  ylab("Tumor thickness (mm)")
```



Training and test sets

There are more patients who lived than died in this dataset. This means there is **class imbalance** for the outcome of interest.

- Use the **stratified** function from the **splitstackshape** package to ensure the proportion of those who died and who lived is the same for both the training and test sets
- Use 70% of the data for training and 30% for testing

Training and test sets

```
library(splitstackshape)
library(caret)
set.seed(10)

x      <- stratified(melanoma, "y", 0.7, keep.rownames = TRUE)
train_set <- x %>% dplyr::select(-rn)
train_index <- as.numeric(x$rn)
test_set   <- melanoma[-train_index,]
```

Training and test sets

```
dim(train_set)
```

```
## [1] 134 8
```

```
head(train_set)
```

```
##      time status  sex  age  year thickness ulcer    y
##      <num> <fctr> <num> <num> <num>      <num> <num> <fctr>
## 1:  2492  Alive    0   54  1971    1.45    0    0
## 2:  3667  Alive    0   42  1967    3.22    0    0
## 3:  2431  Alive    0   49  1971    0.16    0    0
## 4:  3067  Alive    0   29  1969    0.81    0    0
## 5:  1652  Alive    0   58  1973    1.29    0    0
## 6:  5565  Alive    0   41  1962    2.90    0    0
```

```
dim(test_set)
```

```
## [1] 68 8
```

```
head(test_set)
```

```
##      time status  sex  age  year thickness ulcer  y
## 3      35  Alive    1   41  1977    1.34    0  0
## 5     185   Dead    1   52  1965   12.08    1  1
## 6     204   Dead    1   28  1971    4.84    1  1
## 7     210   Dead    1   77  1972    5.16    1  1
## 10     279   Dead    0   68  1971    7.41    1  1
## 11     295   Dead    0   53  1969    4.19    1  1
```



Model training

The **glm** function is used to fit a logistic regression model.

- Only using the training set to fit the model is what makes this a ML logistic regression model
- The **family = "binomial"** argument specifies logistic regression, rather than, e.g., Poisson regression

```
fit_logit <- glm(y ~ thickness,  
                 data = train_set,  
                 family = "binomial")
```


Making predictions

The **predict** function generates predicted probabilities for the test set using the model trained using the training set.

- The first argument is the model to use for prediction
- The second argument is the test set to use
- The **type = "response"** argument instructs the function to output predicted probabilities

```
p_hat_logit <- predict(fit_logit,  
                      newdata = test_set,  
                      type = "response")  
  
head(p_hat_logit)
```

```
##           3           5           6           7           10           11  
## 0.2076846 0.8534673 0.4186368 0.4412780 0.6019683 0.3737767
```

```
y_hat_logit <- ifelse(p_hat_logit > 0.5, "Dead", "Alive")  
  
head(y_hat_logit)
```

```
##           3           5           6           7           10           11  
## "Alive"  "Dead" "Alive" "Alive"  "Dead" "Alive"
```

Making predictions

Each predicted probability must then be transformed into either a 0 or a 1.

- This is based on a specific threshold – 0.5 is a common threshold
- A probability > 0.5 will be changed to a 1
- A probability ≤ 0.5 will be changed to a 0
- The threshold depends on the clinical context

```
p_hat_logit <- predict(fit_logit,  
                      newdata = test_set,  
                      type = "response")  
  
head(p_hat_logit)
```

```
##           3           5           6           7           10           11  
## 0.2076846 0.8534673 0.4186368 0.4412780 0.6019683 0.3737767
```

```
y_hat_logit <- ifelse(p_hat_logit > 0.5, "Dead", "Alive")  
  
head(y_hat_logit)
```

```
##           3           5           6           7           10           11  
## "Alive"  "Dead" "Alive" "Alive"  "Dead" "Alive"
```

The **caret** package

The **caret** package (**C**lassification **A**nd **RE**gression **T**raining) is a set of functions that streamline the process for creating predictive models. The package contains tools for:

- Data splitting
- Pre-processing
- Feature selection
- Model tuning using resampling
- Variable importance estimation



The confusion matrix

The **confusionMatrix** function from the **caret** package outputs the confusion matrix and several performance metrics for the model.

- **data**: the predicted labels
- **reference**: the actual ground truth labels
- **positive**: the value to use as the “positive” or “1” class

The confusion matrix

```
library(caret)
confusionMatrix(data = as.factor(y_hat_logit),
                 reference = test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##              Accuracy : 0.7059
##              95% CI : (0.5829, 0.8102)
##      No Information Rate : 0.6618
##      P-Value [Acc > NIR] : 0.2636727
##
##              Kappa : 0.2084
##
##      Mcnemar's Test P-Value : 0.0007962
```

```
##              Sensitivity : 0.21739
##              Specificity : 0.95556
##              Pos Pred Value : 0.71429
##              Neg Pred Value : 0.70492
##              Prevalence : 0.33824
##              Detection Rate : 0.07353
##      Detection Prevalence : 0.10294
##              Balanced Accuracy : 0.58647
##
##              'Positive' Class : Dead
##
```

The specificity is very high (95.6%), but the sensitivity is very low (21.7%). This means the model is better at predicting patients who will not die.

ROC

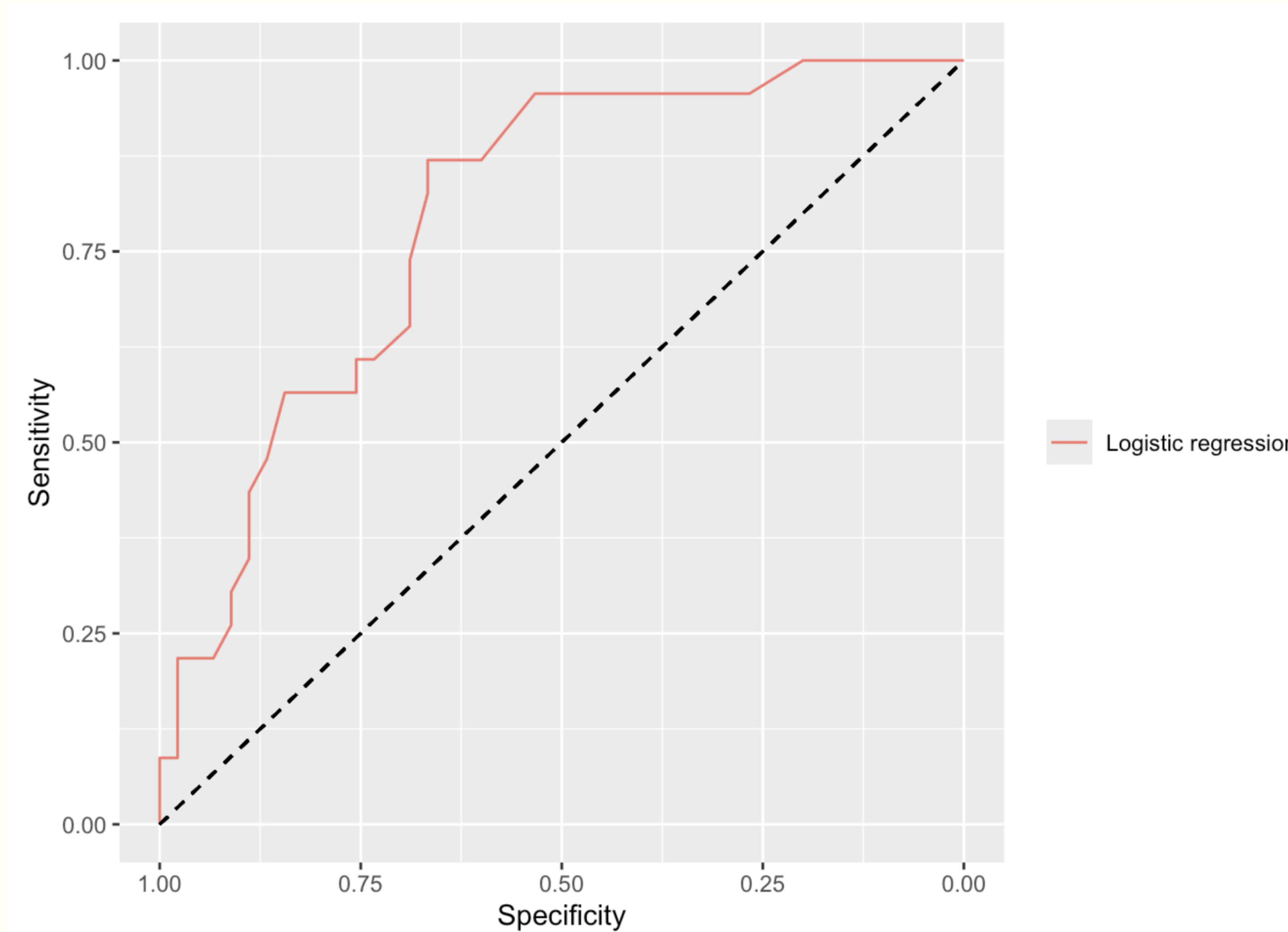
The ROC plot can be created using the **roc** and **ggroc** functions from the **pROC** package.

- The **roc** function calculates the sensitivity and specificity for various classification thresholds
- The **ggroc** function works like **ggplot** to plot the ROC curve

```
library(pROC)
roc_logit <- roc(test_set$y, p_hat_logit)
```

```
ggroc(list("Logistic regression" = roc_logit)) +
  theme(legend.title = element_blank()) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1),
               color = "black",
               linetype = "dashed") +
  ylab("Sensitivity") +
  xlab("Specificity")
```


ROC



AUROC

The AUROC can be calculated using the **auc** function from the **pROC** package.

- This AUROC suggests the model has a reasonable ability to distinguish between individuals who died and those who lived
- A larger sample size might increase this value, especially given the class imbalance

```
auc(roc_logit)
```

```
## Area under the curve: 0.7942
```

Summary

- Logistic regression models the probability of a binary outcome using a logistic (sigmoid) link function.
- Outputs represent predicted probabilities, which are converted to classes using a decision threshold.
- Model performance can be assessed with confusion matrix–based metrics, ROC curves, and AUROC.
- Widely used in healthcare because it is interpretable, robust, and performs well with structured/tabular data.